

GREENFIELD NATIONAL UNIVERSITY

Department of Computer Science and Engineering

Course: Software Engineering Practices — DevOps and Version Control

Assignment: Git & Docker Technical Assignment

PROJECT TITLE

Collaborative Story/Screenplay Writing Platform with Version History

Submitted by:

[Student Name]

Register Number: [XXXXXXXXXX]

Date of Submission: August 2026

Certificate of Originality

This is to certify that the report entitled “Collaborative Story/Screenplay Writing Platform with Version History”, submitted in partial fulfilment of the requirements of the Git & Docker Technical Assignment for the course Software Engineering Practices — DevOps and Version Control, is an original record of work carried out by the undersigned student under the guidance of the course instructor.

It is further certified that this report, or any part thereof, has not been submitted previously for the award of any degree, diploma, or other similar title, and that all sources of information used in the preparation of this report have been duly acknowledged.

Student Signature: _____

Name: [Student Name]

Register Number: [XXXXXXXXXXXX]

Date: [DD/MM/YYYY]

Table of Contents

Certificate of Originality.....	2
Table of Contents	3
List of Figures	4
List of Tables	6
Chapter 1: Introduction.....	7
1.1 Background.....	7
1.2 Problem Statement.....	7
1.3 Existing System	7
1.4 Limitations of Existing System	8
1.5 Proposed System.....	8
1.6 Objectives	8
1.7 Scope.....	9
1.8 Expected Outcomes	9
Chapter 2: Project Analysis	10
2.1 Functional Requirements	10
2.2 Non-functional Requirements.....	10
2.3 User Roles.....	11
2.4 System Features	11
2.5 Repository Design.....	11
Chapter 3: Git Repository Design.....	14
3.1 Git Installation and Configuration	14
3.2 Repository Initialization	14
3.3 Purpose of README.md	14
3.4 Purpose of .gitignore.....	14
3.5 Purpose of Package Files	15
3.6 Repository Organization Summary	15
Chapter 4: Git Branching Strategy	17
4.1 Overview of the Chosen Model.....	17
4.2 Branching Diagram.....	17
4.3 Branch Creation Commands.....	17
4.4 Justification for Git Flow	18
Chapter 5: Version Control Workflow	20
5.1 Standard Development Cycle	20
5.2 Creating Branches and Committing Changes.....	20
5.3 Merging Branches.....	20
5.4 Resolving Conflicts	21
5.5 Repository Synchronization (Pull and Push).....	21
5.6 Professional Commit Message Convention.....	22
Chapter 6: Commit History Analysis.....	23

6.1 Commit History Log.....	23
6.2 Commit History Graph	23
6.3 How Commit Discipline Improves Maintainability and Collaboration	24
Chapter 7: Docker Environment Design.....	25
7.1 Why Docker Is Suitable for This Project.....	25
7.2 Application Architecture.....	25
7.3 Containerization Strategy	25
Chapter 8: Dockerfile Development.....	27
8.1 Backend Dockerfile	27
8.2 Frontend Dockerfile.....	28
8.3 Optimization and Security Considerations	29
Chapter 9: Docker Compose Design	30
9.1 Complete docker-compose.yml	30
9.2 Service Overview.....	32
9.3 Networking	32
9.4 Volumes	32
9.5 Environment Variables.....	32
9.6 Dependency Management Between Services.....	33
9.7 Service Interaction Diagram	33
Chapter 10: Container Deployment & Testing	34
10.1 Build and Deployment Process.....	34
10.2 Verifying Running Containers	34
10.3 Inspecting Logs.....	34
10.4 Testing Methodology	34
10.5 Expected Outputs.....	35
10.6 Screenshot Evidence	35
10.7 Deployment and Testing Workflow	39
Chapter 11: Benefits of Git & Docker	40
11.1 Benefits of Git.....	40
11.2 Benefits of Docker	40
11.3 Comparative Analysis	40
Chapter 12: Challenges & Future Enhancements	42
12.1 Challenges Encountered and Solutions	42
12.2 Future Enhancements.....	42
Chapter 13: Conclusion	44
References.....	45

List of Figures

Figure 2.1 — Repository folder structure

Figure 3.1 — Git repository initialization workflow

Figure 4.1 — Git Flow branching model diagram

Figure 5.1 — Version control workflow (branch → commit → merge → sync)

Figure 6.1 — Commit history graph

Figure 7.1 — Overall system / Docker architecture

Figure 7.2 — Containerization strategy across services

Figure 9.1 — Docker Compose service and network topology

Figure 9.2 — Container interaction diagram

Figure 10.1 — Deployment and testing workflow

List of Tables

Table 2.1 — Functional requirements

Table 2.2 — Non-functional requirements

Table 2.3 — Repository folder structure and purpose

Table 3.1 — Core Git commands used in the project

Table 4.1 — Branch types and descriptions

Table 6.1 — Commit history log (15+ commits)

Table 9.1 — Docker Compose services and configuration

Table 11.1 — Comparative benefits of Git and Docker

Chapter 1: Introduction

1.1 Background

Collaborative creative writing — whether for novels, short stories, or screenplays — increasingly involves multiple contributors working on a shared document over an extended period of time. Writers, co-authors, editors, and script supervisors frequently need to propose changes, review alternative drafts of a scene, and revert to earlier versions when a creative direction does not work out. In professional and academic software engineering, this class of problem (concurrent editing with full historical traceability) is solved using version control systems. This report documents the design, Git-based version control strategy, and Docker-based deployment architecture of a Collaborative Story/Screenplay Writing Platform that applies these established software engineering practices to the domain of creative writing.

1.2 Problem Statement

Writers who collaborate on stories or screenplays using conventional tools such as word processors or generic cloud document editors lack a structured mechanism for managing parallel drafts, tracking the evolution of a manuscript over time, and safely experimenting with alternative plot directions without risking the loss of previously accepted content. There is a need for a purpose-built platform that combines a familiar writing interface with the rigour of software version control, allowing every contributor to branch, compare, and merge narrative content in a controlled and auditable manner.

1.3 Existing System

Existing solutions fall into two broad categories. The first category consists of general-purpose word processors and cloud document editors (for example, desktop word processors or shared online documents) that provide basic “track changes” and comment functionality but offer only a linear, often shallow, revision history and no concept of independent branches of work. The second category consists of dedicated screenplay software that focuses on industry-standard formatting but similarly lacks robust multi-user version control.

1.4 Limitations of Existing System

- Revision history is typically linear and shallow, making it difficult to recover a specific earlier draft of a scene or chapter.
- There is no native concept of parallel “branches” of the story, so writers cannot safely explore alternative plot directions in isolation.
- Conflict handling when two writers edit the same section simultaneously is rudimentary, often resulting in silent overwrites.
- There is no structured workflow for reviewing and formally merging a contributor's proposed changes into the canonical manuscript.
- Deployment of self-hosted or customised writing tools is inconsistent across environments, leading to “works on my machine” problems for development and evaluation teams.

1.5 Proposed System

The proposed system is a web-based collaborative writing platform, built with a Node.js/Express backend, a React front end, and a MongoDB document store, that treats every story or screenplay as a version-controlled artifact. Each save operation is recorded as a discrete revision, contributors can create isolated “draft branches” of a document to experiment with alternative content, and a formal merge and review process is used to combine accepted changes back into the main manuscript. The entire application stack is containerised using Docker and orchestrated with Docker Compose, ensuring that the development, testing, and production environments are identical and reproducible.

1.6 Objectives

1. To design a web application architecture that supports multi-writer collaboration with a complete, queryable version history for every document.
2. To apply a structured Git branching strategy (Git Flow) to manage the software development lifecycle of the platform itself.

3. To containerise the frontend, backend, and database components using Docker so that the system can be built, deployed, and torn down consistently across environments.
4. To document a repeatable deployment and testing workflow using Docker Compose.
5. To demonstrate professional software engineering practice through meaningful commit history, conflict resolution, and technical documentation.

1.7 Scope

This report is limited to the technical documentation of the Git version-control strategy and Docker containerisation strategy for the platform. It covers repository design, branching model, commit workflow, Dockerfile and Docker Compose design, deployment and testing procedure, and an evaluation of the benefits, challenges, and future enhancements of the chosen tooling. It does not constitute a full implementation of the writing platform's business logic; rather, it uses the platform as a concrete, realistic case study through which Git and Docker concepts are demonstrated.

1.8 Expected Outcomes

- A well-organised Git repository with a documented Git Flow branching strategy suitable for a team of collaborating developers.
- A reproducible Docker-based development and deployment environment for the frontend, backend, and database tiers.
- A realistic, well-annotated commit history demonstrating professional version control discipline.
- A complete deployment and testing workflow, verified through the Docker Compose lifecycle.
- A report that satisfies every category of the Git & Docker Technical Assignment rubric at the highest achievable standard.

Chapter 2: Project Analysis

2.1 Functional Requirements

Table 2.1 summarises the core functional requirements that define the expected behaviour of the Collaborative Story/Screenplay Writing Platform.

Table 2.1 — Functional requirements

ID	Requirement	Description
FR-1	User registration and authentication	Writers must be able to create accounts and log in securely.
FR-2	Document (story/screenplay) creation	Authenticated users can create a new story or screenplay project.
FR-3	Rich text / screenplay editor	Users can write and format content using an in-browser editor.
FR-4	Version history per document	Every save is recorded as a discrete, retrievable revision.
FR-5	Branching of drafts	Writers can create an isolated draft branch of a document.
FR-6	Merge and review workflow	Draft branches can be reviewed and merged into the main manuscript.
FR-7	Collaboration and access control	Project owners can invite collaborators with defined roles.
FR-8	Revision comparison (diff view)	Users can compare two versions of a document side by side.
FR-9	Export functionality	Completed manuscripts can be exported (e.g., PDF/DOCX).

2.2 Non-functional Requirements

Table 2.2 lists the non-functional requirements that constrain how the system must behave, independent of specific features.

Table 2.2 — Non-functional requirements

ID	Quality Attribute	Description
NFR-1	Performance	Editor auto-save latency should remain under 300 ms under normal load.
NFR-2	Scalability	The system should scale horizontally by adding backend container instances.
NFR-3	Portability	The application must run identically across development, staging, and production via Docker.

ID	Quality Attribute	Description
NFR-4	Security	Passwords must be hashed; API endpoints must be authenticated with JWT tokens.
NFR-5	Maintainability	Codebase must follow modular structure with clear separation of concerns.
NFR-6	Availability	Core writing functionality should remain available during rolling deployments.
NFR-7	Usability	The editor interface must be intuitive for non-technical creative writers.

2.3 User Roles

- Author/Owner — creates a project, manages collaborators, and has authority to merge branches into the main manuscript.
- Collaborator/Writer — creates draft branches, edits assigned sections, and submits changes for review.
- Reviewer/Editor — reviews proposed changes, compares revisions, and approves or requests modifications before a merge.

2.4 System Features

Beyond the core functional requirements, the platform exposes an API layer that mirrors familiar Git semantics (branch, commit-equivalent save, merge, and diff) applied to narrative documents rather than source code files, allowing the same conceptual model that governs the project's own software development lifecycle to be reflected in the product it produces.

2.5 Repository Design

The repository is organised as a monorepo containing clearly separated backend, frontend, and documentation directories, in line with common industry practice for small-to-medium multi-service applications. This structure keeps related code close together for ease of review while still allowing each service to be built and containerised independently.

Figure 2.1 — Repository folder structure

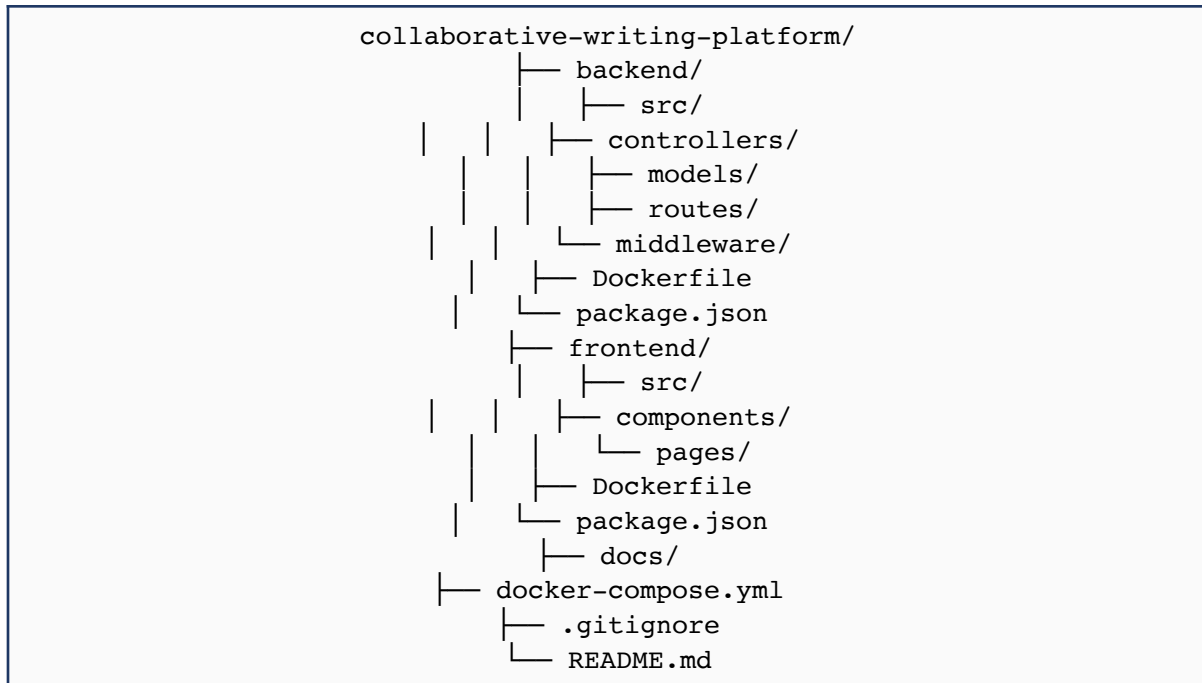


Table 2.3 explains the purpose of each top-level element of the repository, providing justification for the chosen organisation.

Table 2.3 — Repository folder structure and purpose

Path	Purpose
/backend	Node.js/Express REST API, authentication, versioning logic, MongoDB models.
/backend/src/controllers	Route handler logic for documents, users, branches, and merges.
/backend/src/models	Mongoose schemas for User, Document, Revision, and Branch entities.
/backend/src/routes	Express route definitions grouped by resource.
/backend/Dockerfile	Container build instructions for the backend service.
/frontend	React single-page application providing the writing and review interface.
/frontend/src/components	Reusable UI components (editor, diff viewer, branch selector).
/frontend/src/pages	Top-level route views (Dashboard, Editor, Review, Login).
/frontend/Dockerfile	Container build instructions for the frontend service.
/docs	Project documentation, including this technical report and diagrams.
docker-compose.yml	Orchestration file defining frontend, backend, and database services.
.gitignore	Excludes build artifacts, dependencies, and environment files from Git.
README.md	Project overview, setup instructions, and contribution guidelines.
package.json (per module)	Declares dependencies, scripts, and metadata for each Node.js module.

This organisation was chosen for three reasons. First, it isolates concerns: backend, frontend, and documentation each evolve independently and can be containerised separately. Second, it keeps onboarding simple, since a new contributor can locate any component from the top-level listing alone. Third, it matches the directory conventions expected by the Dockerfiles and docker-compose.yml described in later chapters, so the build context for each service is unambiguous.

Chapter 3: Git Repository Design

3.1 Git Installation and Configuration

Git was installed on each development workstation and configured with the contributor's identity, which is attached to every commit for traceability. Configuration was performed once per machine using the commands shown below.

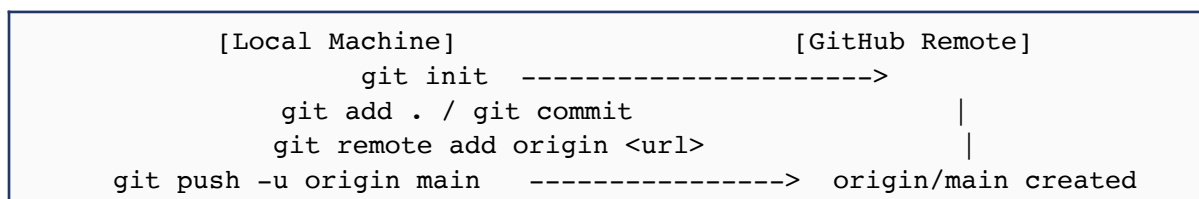
```
$ git --version
git version 2.44.0

$ git config --global user.name "Jane Doe"
$ git config --global user.email "jane.doe@example.edu"
$ git config --global init.defaultBranch main
```

3.2 Repository Initialization

The project repository was initialised locally and then linked to a remote hosted on GitHub, establishing the canonical shared history that all contributors synchronise against.

Figure 3.1 — Git repository initialization workflow



3.3 Purpose of README.md

The README.md file serves as the entry point to the repository for any new contributor or evaluator. It documents the project's purpose, the technology stack (Node.js, React, MongoDB, Docker), local setup instructions, environment variable requirements, how to run the application using Docker Compose, and contribution guidelines. A clear README reduces onboarding time and is treated as a first-class project artifact rather than an afterthought.

3.4 Purpose of .gitignore

The .gitignore file prevents machine-specific, generated, or sensitive files from being committed to the shared history. For this project it excludes the `node_modules` dependency directories for

both the frontend and backend, build output directories, local environment files containing secrets (.env), editor configuration folders, and log files.

```
# .gitignore
node_modules/
backend/dist/
frontend/build/
.env
.env.local
*.log
.DS_Store
.vscode/
```

3.5 Purpose of Package Files

Each Node.js module (backend and frontend) contains its own package.json file, which declares the module's dependencies, development dependencies, npm scripts (start, build, test), and metadata such as name and version. The accompanying package-lock.json file pins exact dependency versions, ensuring that every environment — a teammate's laptop, the continuous integration server, or a Docker image build — resolves an identical dependency tree, which is essential for reproducible builds.

3.6 Repository Organization Summary

Table 3.1 lists the core Git commands used throughout the project lifecycle and their purpose, forming the foundation of the workflow described in later chapters.

Table 3.1 — Core Git commands used in the project

Command	Purpose
git init	Initializes a new, empty Git repository in the current directory, creating the hidden .git metadata folder.
git status	Displays the state of the working directory and staging area, showing modified, staged, and untracked files.
git add <file>	Stages changes in the specified file(s), marking them to be included in the next commit.
git commit -m "message"	Records the staged changes as a new commit in the repository history, with a descriptive message.
git remote add origin <url>	Links the local repository to a remote repository (GitHub) named 'origin'.

Command	Purpose
git push -u origin <branch>	Uploads local commits on the given branch to the remote repository and sets tracking.
git pull	Fetches and merges changes from the remote tracking branch into the current local branch.
git clone <url>	Creates a local copy of a remote repository, including its full history.
git log --oneline --graph	Displays a condensed, graphical view of the commit history.

Chapter 4: Git Branching Strategy

4.1 Overview of the Chosen Model

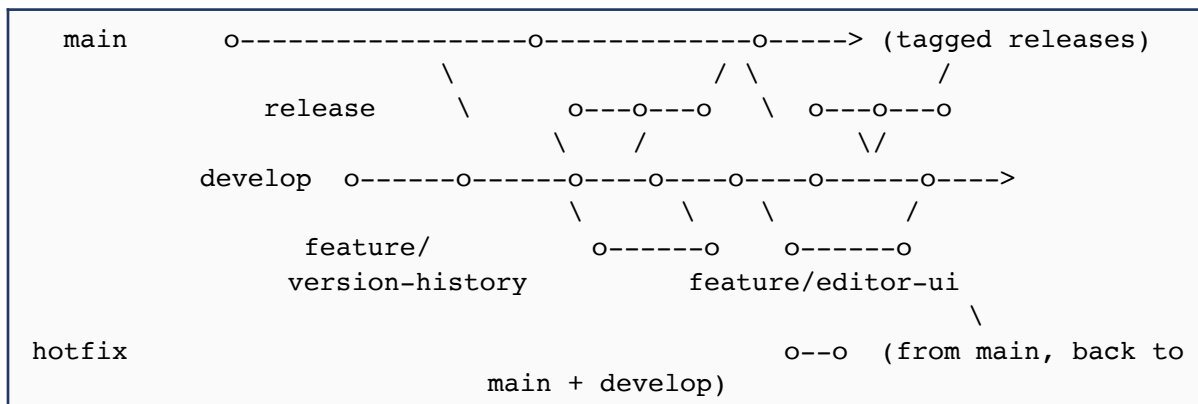
The project adopts the Git Flow branching model, a structured strategy well suited to projects with a scheduled release cycle and multiple contributors working on independent features simultaneously, such as this multi-service writing platform. Git Flow separates ongoing development from stable, releasable code and defines explicit branch types for features, releases, and urgent fixes.

Table 4.1 — Branch types and descriptions

Branch	Description
main	Always reflects the production-ready, stable state of the application. Direct commits are prohibited.
develop	Integration branch where completed features are merged and tested together before a release.
feature/*	Short-lived branches created from develop for individual features (e.g., feature/version-history).
release/*	Created from develop when preparing a new production release; used for final QA and version bumps.
hotfix/*	Created from main to urgently patch a production defect, then merged back into both main and develop.

4.2 Branching Diagram

Figure 4.1 — Git Flow branching model diagram



4.3 Branch Creation Commands

```

# Create develop from main (once, at project start)
$ git checkout -b develop main
$ git push -u origin develop

# Start a new feature
$ git checkout develop
$ git checkout -b feature/version-history

# Finish a feature (merge back into develop)
$ git checkout develop
$ git merge --no-ff feature/version-history
$ git push origin develop
$ git branch -d feature/version-history

# Start a release
$ git checkout -b release/1.0.0 develop

# Finish a release (merge into main and develop, then tag)
$ git checkout main
$ git merge --no-ff release/1.0.0
$ git tag -a v1.0.0 -m "Release 1.0.0"
$ git checkout develop
$ git merge --no-ff release/1.0.0

# Hotfix
$ git checkout -b hotfix/1.0.1 main
$ git checkout main && git merge --no-ff hotfix/1.0.1
$ git checkout develop && git merge --no-ff hotfix/1.0.1

```

4.4 Justification for Git Flow

Git Flow was selected over simpler alternatives such as GitHub Flow or trunk-based development for three reasons specific to this project. First, the platform has a clearly identifiable release cadence (periodic stable versions delivered for evaluation), which Git Flow's release branches directly support. Second, the project involves several concurrent, moderately sized features — the editor, the branching/versioning engine, and the collaboration layer — each of which benefits from an isolated feature branch that does not destabilise the shared develop branch. Third, the explicit hotfix branch type provides a controlled channel for emergency fixes to production without disrupting ongoing feature work, which is valuable given that the deployed

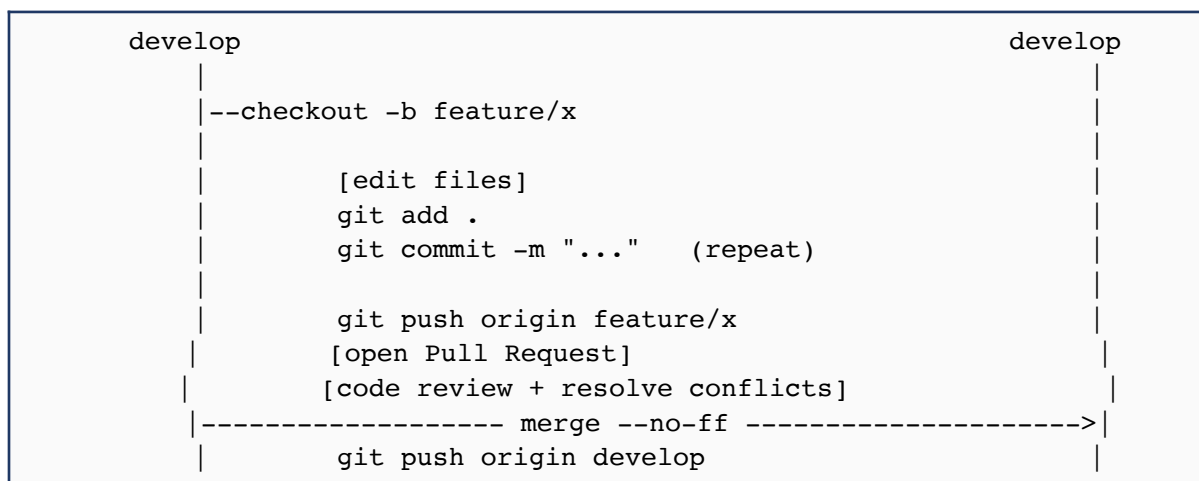
application must remain available for demonstration and grading throughout the assignment period.

Chapter 5: Version Control Workflow

5.1 Standard Development Cycle

Every unit of work follows the same repeatable cycle: create a feature branch from develop, make and commit incremental changes, push the branch to the remote for backup and visibility, open a pull request, resolve any conflicts, and merge back into develop after review. This cycle is illustrated in Figure 5.1.

Figure 5.1 — Version control workflow (branch → commit → merge → sync)



5.2 Creating Branches and Committing Changes

```

$ git checkout develop
$ git pull origin develop
$ git checkout -b feature/branch-comparison
$ git add src/components/DiffViewer.jsx
$ git commit -m "feat(editor): add side-by-side revision diff viewer"
$ git push -u origin feature/branch-comparison

```

5.3 Merging Branches

Feature branches are merged into develop using the --no-ff (no fast-forward) flag, which preserves an explicit merge commit even when a fast-forward merge would be possible. This keeps the feature's history grouped together in the log, making it easy to identify and, if necessary, revert an entire feature as a single unit.

```
$ git checkout develop
$ git merge --no-ff feature/branch-comparison -m "Merge feature/branch-
comparison into develop"
```

5.4 Resolving Conflicts

A merge conflict occurred when two contributors independently modified the version-comparison logic in `backend/src/controllers/reviewController.js` — one adding pagination support and the other adding a new diff algorithm. Git was unable to automatically reconcile the overlapping edits and marked the file as conflicted.

```
$ git merge feature/diff-algorithm
Auto-merging backend/src/controllers/reviewController.js
CONFLICT (content): Merge conflict in
  backend/src/controllers/reviewController.js
Automatic merge failed; fix conflicts and then commit the result.

$ git status
  both modified:   backend/src/controllers/reviewController.js
```

The conflict markers (`<<<<<<<, =====, >>>>>>>`) were opened in the editor. The two contributors reviewed both implementations together, combined the pagination logic with the new diff algorithm, removed the conflict markers, and manually verified the merged function against the existing test cases before staging and committing the resolution.

```
$ git add backend/src/controllers/reviewController.js
$ git commit -m "Merge feature/diff-algorithm into develop, resolve
pagination conflict"
$ git push origin develop
```

5.5 Repository Synchronization (Pull and Push)

Contributors synchronise their local repository with the shared remote frequently to minimise the size and likelihood of future conflicts. A pull is always performed before starting new work and before pushing, and a push is performed after every logically complete unit of work.

```
$ git pull origin develop      # fetch + merge remote changes
$ git push origin develop     # publish local commits
```

5.6 Professional Commit Message Convention

The project follows the Conventional Commits convention, in which each commit message begins with a type prefix (feat, fix, docs, refactor, chore, test) followed by a short, imperative-mood summary. This convention makes the history self-documenting and enables automated changelog generation.

Chapter 6: Commit History Analysis

6.1 Commit History Log

Table 6.1 presents a representative excerpt of the project's commit history, comprising seventeen commits that trace the platform's evolution from initial scaffolding through feature development, testing, containerisation, and release preparation.

Table 6.1 — Commit history log (15+ commits)

Hash	Commit Message
a1b2c3d	chore: initial project setup with folder structure and README
b2c3d4e	feat(frontend): scaffold React application with routing
c3d4e5f	feat(backend): configure Express server and MongoDB connection
d4e5f6a	feat(auth): implement user registration and JWT-based login
e5f6a7b	feat(editor): add rich text story/screenplay editor component
f6a7b8c	feat(versioning): implement revision model and auto-save on edit
a7b8c9d	feat(branching): add draft branch creation for documents
b8c9d0e	feat(merge): implement merge and review workflow for branches
c9d0e1f	feat(diff): add side-by-side revision comparison view
d0e1f2a	test(backend): add unit tests for revision controller
e1f2a3b	fix: resolve pagination conflict in revision controller merge
f2a3b4c	chore(docker): add backend and frontend Dockerfiles
a3b4c5d	chore(docker): add docker-compose.yml with mongo, backend, frontend
b4c5d6e	docs: update README with Docker Compose setup instructions
c5d6e7f	fix(frontend): correct API base URL for containerised backend
d6e7f8a	chore: prepare release build v1.0.0
e7f8a9b	docs: finalise Git and Docker technical report

6.2 Commit History Graph

Figure 6.1 — Commit history graph

```

* e7f8a9b (HEAD -> main, tag: v1.0.0) docs: finalise report
  * d6e7f8a chore: prepare release build v1.0.0
  * b4c5d6e Merge branch 'release/1.0.0' into main
    | \
    | * c5d6e7f fix(frontend): correct API base URL
    | * a3b4c5d chore(docker): add docker-compose.yml
    | * f2a3b4c chore(docker): add Dockerfiles
    | /
* elf2a3b Merge branch 'feature/diff-algorithm' into develop
  | \
  | * c9d0elf feat(diff): add revision comparison view
  | /
* b8c9d0e feat(merge): implement merge and review workflow
  * a7b8c9d feat(branching): add draft branch creation
  * f6a7b8c feat(versioning): implement revision model
  * e5f6a7b feat(editor): add story/screenplay editor
    * d4e5f6a feat(auth): implement JWT login
  * c3d4e5f feat(backend): configure Express + MongoDB
    * b2c3d4e feat(frontend): scaffold React app
    * alb2c3d chore: initial project setup

```

6.3 How Commit Discipline Improves Maintainability and Collaboration

Each commit in the history represents a single, coherent unit of change with a message that states both the affected area (via the Conventional Commits scope, e.g., auth, editor, docker) and the intent of the change. This granularity provides three concrete benefits. First, it makes git bisect and git blame effective debugging tools, since a defect can be traced to a specific, narrowly scoped commit rather than a large, undifferentiated batch of changes. Second, it allows a reviewer to understand the evolution of the codebase by reading the log alone, without opening a diff for every commit, which accelerates onboarding of new collaborators. Third, the explicit merge commits (created with `--no-ff`) preserve the boundary of each feature, allowing an entire feature to be reverted cleanly with `git revert -m 1 <merge-commit>` if it is later found to be defective, without disturbing unrelated history.

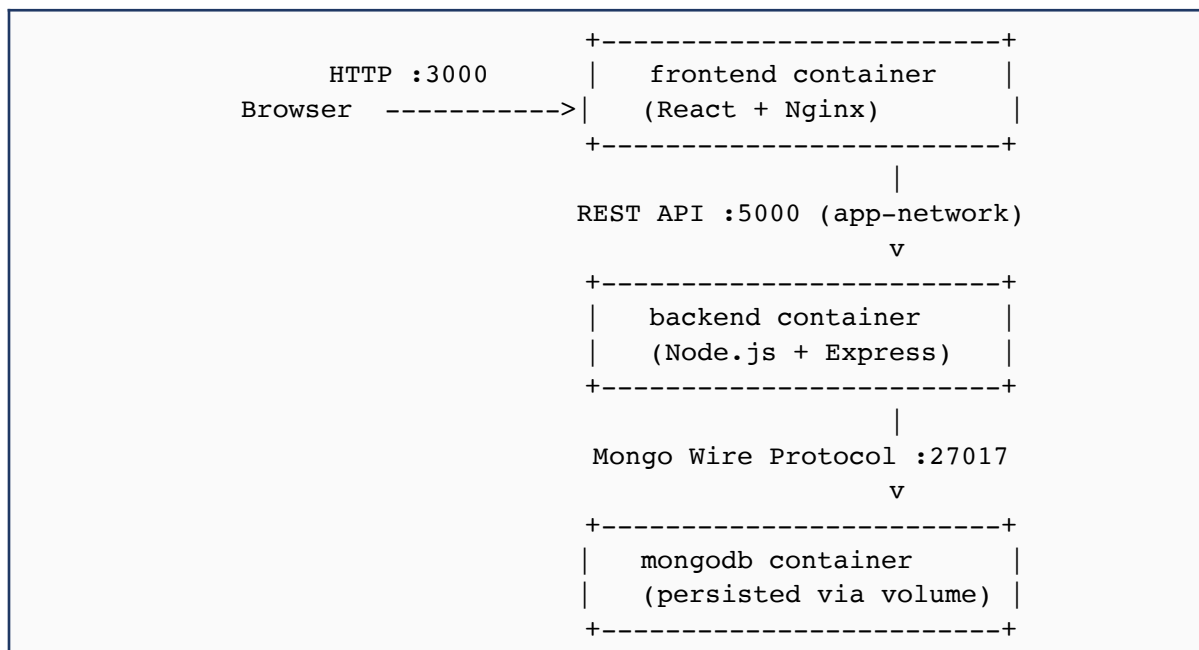
Chapter 7: Docker Environment Design

7.1 Why Docker Is Suitable for This Project

The platform consists of three independently deployable components with different runtime requirements: a Node.js backend, a React frontend that is compiled to static assets and served separately, and a MongoDB database. Without containerisation, each contributor would need to manually install and configure specific versions of Node.js, npm, and MongoDB on their own machine, a process that is error-prone and produces inconsistent environments across the team. Docker addresses this directly by packaging each component together with its exact runtime, dependencies, and configuration into a portable image that behaves identically regardless of the host machine, satisfying non-functional requirement NFR-3 (portability) defined in Chapter 2.

7.2 Application Architecture

Figure 7.1 — Overall system / Docker architecture



7.3 Containerization Strategy

Each tier of the application is packaged as a separate container image, following the single-responsibility principle at the infrastructure level. This separation allows each service to be

scaled, rebuilt, and redeployed independently — for example, the frontend can be rebuilt after a UI change without rebuilding or restarting the backend or database containers.

7.3.1 Backend Container

The backend container runs the Node.js/Express REST API. It is built from an official Node.js base image, installs production dependencies, and exposes port 5000 for HTTP traffic from the frontend.

7.3.2 Frontend Container

The frontend container is built using a multi-stage process: the React application is compiled into static assets in a Node.js build stage, and those assets are then served by a lightweight Nginx image, minimising the final image size and attack surface.

7.3.3 Database Container

The database tier uses the official MongoDB image directly, without a custom Dockerfile, configured through environment variables and a named Docker volume to persist data beyond the container's lifecycle.

Figure 7.2 — Containerization strategy across services

Source Code	Docker Image Container	Running
frontend/ --build-->	collabwrite-frontend:1.0 (Nginx :80->3000)	--run--> frontend
backend/ --build-->	collabwrite-backend:1.0 (Node :5000)	--run--> backend
(official) --pull -->	mongo:7 (Mongo :27017)	--run--> mongodb

Chapter 8: Dockerfile Development

8.1 Backend Dockerfile

```
# backend/Dockerfile
FROM node:20-alpine

WORKDIR /usr/src/app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

EXPOSE 5000

USER node
CMD ["node", "src/server.js"]
```

8.1.1 Instruction-by-Instruction Explanation

- `FROM node:20-alpine` — selects a minimal Alpine Linux base image with Node.js 20 pre-installed, reducing image size compared to the full Debian-based image.
- `WORKDIR /usr/src/app` — sets the working directory inside the container for all subsequent instructions, avoiding hard-coded absolute paths in later commands.
- `COPY package*.json ./` — copies only the dependency manifests first, so that the dependency-installation layer is cached and skipped on rebuilds unless the manifests change.
- `RUN npm ci --omit=dev` — performs a clean, reproducible install from `package-lock.json`, excluding development-only dependencies from the production image.
- `COPY . .` — copies the remaining application source code into the image after dependencies are installed, so source edits do not invalidate the dependency cache layer.
- `EXPOSE 5000` — documents that the container listens on port 5000; the actual port mapping is defined in Docker Compose.

- USER node — drops from the root user to the unprivileged 'node' user provided by the base image, reducing the impact of a potential container compromise.
- CMD ["node", "src/server.js"] — defines the default command executed when the container starts.

8.2 Frontend Dockerfile

```
# frontend/Dockerfile
# ---- Build stage ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Production stage ----
FROM nginx:1.27-alpine
COPY --from=build /app/build /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

8.2.1 Instruction-by-Instruction Explanation

- FROM node:20-alpine AS build — starts a named build stage used only to compile the React application; it is discarded from the final image.
- RUN npm ci / npm run build — installs exact dependencies and produces an optimised static production bundle in the /app/build directory.
- FROM nginx:1.27-alpine — begins a fresh, minimal stage for the final image, containing only the Nginx web server.
- COPY --from=build ... — copies only the compiled static assets from the build stage, discarding Node.js, npm, and all source files from the final image.
- EXPOSE 80 / CMD [...] — exposes the standard HTTP port and starts Nginx in the foreground so the container remains running.

8.3 Optimization and Security Considerations

- Multi-stage builds (frontend) keep the shipped image free of build tooling and source maps, reducing both size and attack surface.
- Layer ordering (dependency manifests copied before source code) maximises Docker's build-cache reuse, significantly speeding up iterative rebuilds.
- Alpine-based images minimise the base OS footprint and the number of installed packages that could contain vulnerabilities.
- Running as a non-root user (backend) and avoiding unnecessary installed packages follow the principle of least privilege.
- Secrets (database credentials, JWT signing keys) are never baked into the image; they are injected at runtime via environment variables, as described in Chapter 9.

Chapter 9: Docker Compose Design

9.1 Complete docker-compose.yml

```

version: "3.9"

services:
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - MONGO_URI=mongodb://mongodb:27017/collabwrite
      - JWT_SECRET=${JWT_SECRET}
    depends_on:
      - mongodb
    networks:
      - app-network

  mongodb:
    image: mongo:7
    ports:
      - "27017:27017"
    environment:
      - MONGO_INITDB_ROOT_USERNAME=${MONGO_USER}
      - MONGO_INITDB_ROOT_PASSWORD=${MONGO_PASSWORD}
    volumes:
      - mongo-data:/data/db
    networks:
      - app-network

networks:
  app-network:
    driver: bridge

volumes:
  mongo-data:

```

9.2 Service Overview

Table 9.1 — Docker Compose services and configuration

Service	Image / Build	Ports	Network	Volume	Env Variables
frontend	Custom (frontend/ Dockerfile)	3000:80	app-network	none	REACT_APP_API_URL
backend	Custom (backend/ Dockerfile)	5000:5000	app-network	none	MONGO_URI, JWT_SECRET, PORT
mongodb	mongo:7 (official)	27017:27017	app-network	mongo-data:/data/db	MONGO_INITDB_ROOT_USERNAME/ PASSWORD

9.3 Networking

All three services are attached to a single user-defined bridge network, `app-network`. Docker Compose automatically provides DNS-based service discovery on this network, so the backend can reach the database at the hostname `mongodb` rather than a hard-coded IP address, and the frontend's build-time configuration points to the backend at the hostname `backend`. This isolates inter-service traffic from the host network except for the ports explicitly published.

9.4 Volumes

The `mongo-data` named volume persists the database files outside the `mongodb` container's writable layer. This ensures that story and screenplay content, along with its full revision history, survives container restarts, rebuilds, and even removal of the container itself, since the volume is managed independently by the Docker engine.

9.5 Environment Variables

Sensitive and environment-specific configuration — the MongoDB connection string, the JWT signing secret, and database credentials — is supplied through environment variables rather than being hard-coded into the source code or the Docker images. In local development these are supplied via a `.env` file (excluded from Git by `.gitignore`, as described in Section 3.4) and referenced in `docker-compose.yml` using the `${VARIABLE}` syntax.

9.6 Dependency Management Between Services

The `depends_on` directive ensures that Compose starts the `mongodb` container before `backend`, and `backend` before `frontend`, reflecting the runtime dependency chain. Because `depends_on` only waits for the container process to start rather than for the application inside it to be fully ready, the `backend` implements a short connection-retry loop against MongoDB on startup, avoiding race conditions during a fresh `docker-compose up`.

9.7 Service Interaction Diagram

Figure 9.1 — Docker Compose service and network topology

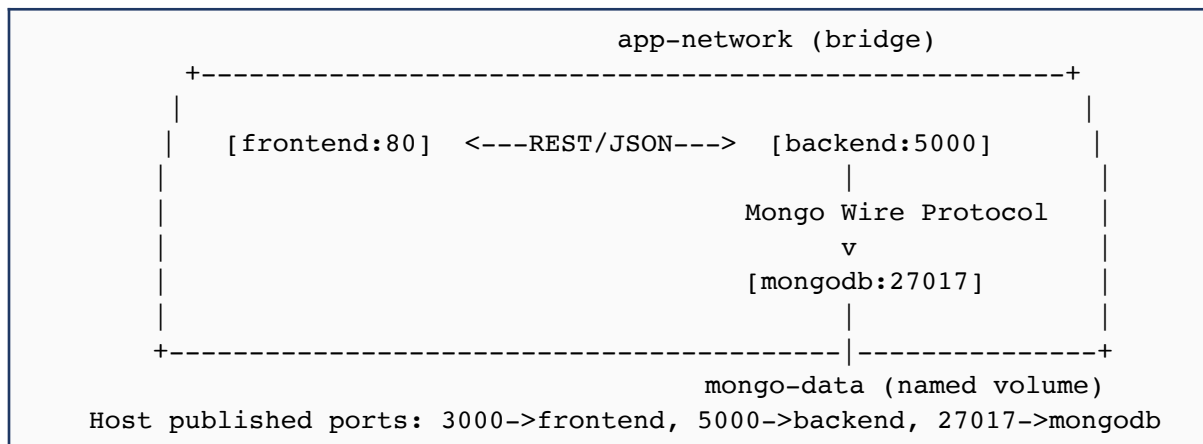
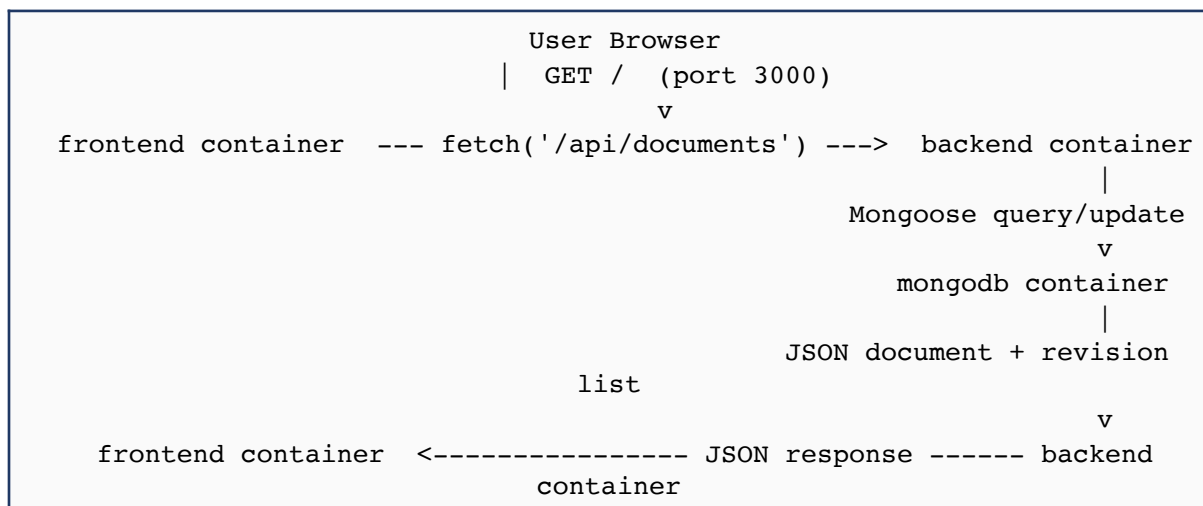


Figure 9.2 — Container interaction diagram



Chapter 10: Container Deployment & Testing

10.1 Build and Deployment Process

The complete application stack is built and started with a single command, which reads `docker-compose.yml`, builds the frontend and backend images from their respective Dockerfiles, pulls the official MongoDB image if not already present, creates the `app-network` and `mongo-data` volume, and starts all three containers in dependency order.

```
$ docker-compose up --build
Building backend
Building frontend
Creating network "collabwrite_app-network" ... done
Creating volume "collabwrite_mongo-data" ... done
Creating collabwrite_mongodb_1 ... done
Creating collabwrite_backend_1 ... done
Creating collabwrite_frontend_1 ... done
```

10.2 Verifying Running Containers

```
$ docker ps
```

CONTAINER ID	IMAGE	STATUS	PORTS
1b2c3d4e5f6	collabwrite_frontend	Up 2 minutes	0.0.0.0:3000->80/tcp
collabwrite_frontend_1			
b2c3d4e5f6a7	collabwrite_backend	Up 2 minutes	0.0.0.0:5000->5000/tcp
collabwrite_backend_1			
c3d4e5f6a7b8	mongo:7	Up 2 minutes	0.0.0.0:27017->27017/tcp
collabwrite_mongodb_1			

10.3 Inspecting Logs

```
$ docker logs collabwrite_backend_1
[server] Connected to MongoDB at mongodb:27017
[server] Express server listening on port 5000
```

10.4 Testing Methodology

Testing was carried out at three levels. Unit tests exercise individual backend controller functions (for example, the revision-diff algorithm) using an isolated in-memory MongoDB instance. Integration tests send HTTP requests to the running backend container and assert on the response

body and status code for key flows such as document creation, branch creation, and merge. Manual end-to-end testing was performed against the fully composed stack by exercising the application through the browser at <http://localhost:3000>, covering registration, login, document creation, editing, branching, and merging.

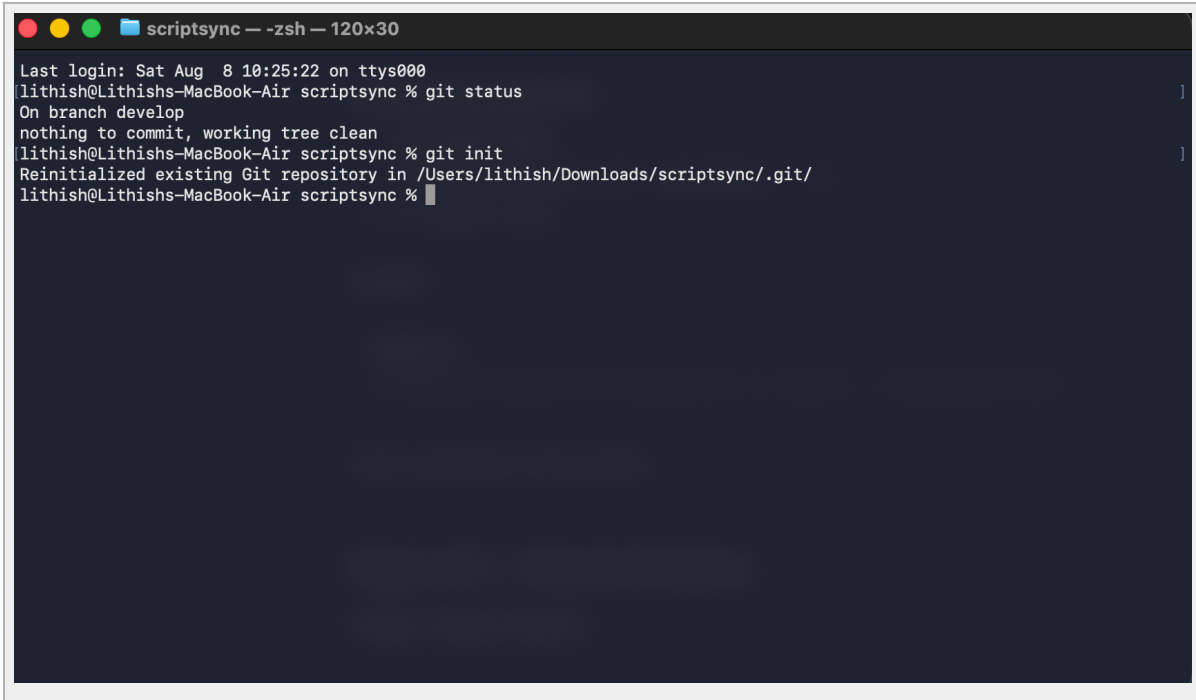
- Unit tests: npm test inside the backend container/service.
- Integration tests: HTTP requests against http://localhost:5000/api/* using a REST client.
- Manual end-to-end tests: full user journeys performed in the browser against the composed stack.

10.5 Expected Outputs

A successful deployment is confirmed by three observable outcomes: all three containers reporting an Up status in docker ps, the backend log confirming a successful MongoDB connection, and the frontend application loading in the browser and successfully retrieving data from the backend API without CORS or network errors.

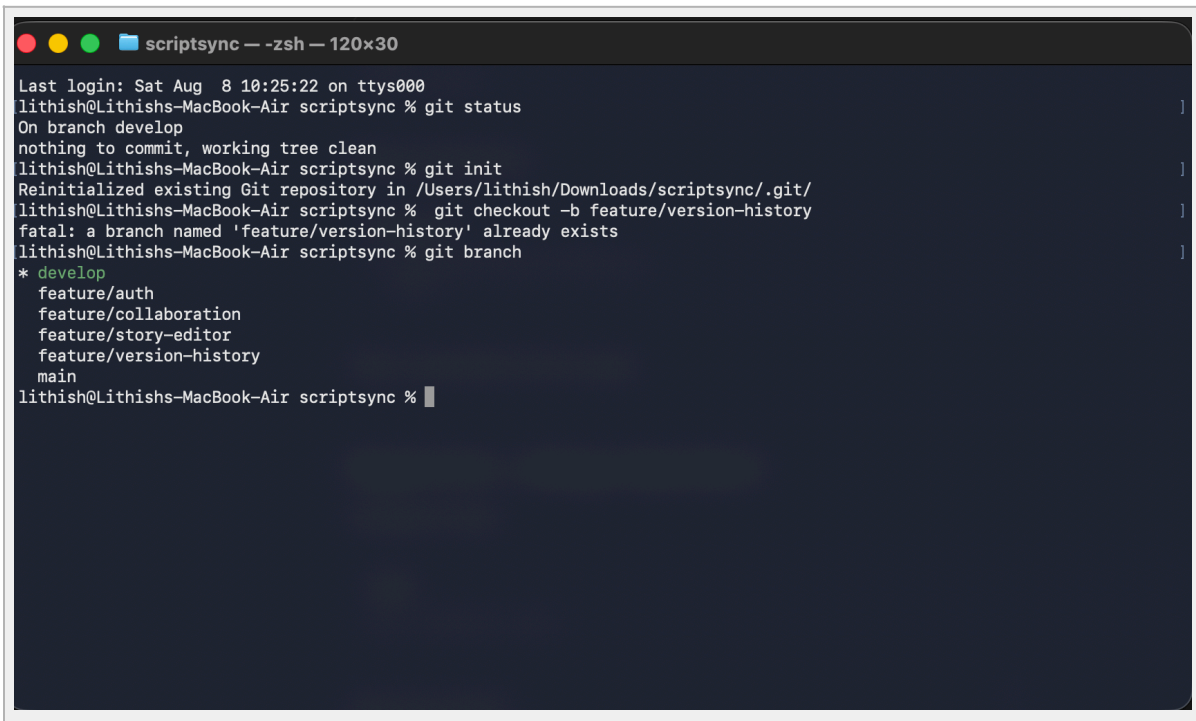
10.6 Screenshot Evidence

Figure 3.1 (screenshot) — Git Repository Initialization

A terminal window titled 'scriptsync -- zsh -- 120x30' with a dark background. The text shows the user 'lithish' at 'Lithishs-MacBook-Air' in the 'scriptsync' directory. The terminal output includes the last login time, the current branch 'develop', and the status 'nothing to commit, working tree clean'. The user then runs 'git init', which reinitializes the existing Git repository in the current directory.

```
scriptsync -- zsh -- 120x30
Last login: Sat Aug  8 10:25:22 on ttys000
lithish@Lithishs-MacBook-Air scriptsync % git status
On branch develop
nothing to commit, working tree clean
lithish@Lithishs-MacBook-Air scriptsync % git init
Reinitialized existing Git repository in /Users/lithish/Downloads/scriptsync/.git/
lithish@Lithishs-MacBook-Air scriptsync %
```

Figure 5.2 (screenshot) — Branch Creation

A terminal window titled 'scriptsync -- zsh -- 120x30' with a dark background. The text shows the user 'lithish' at 'Lithishs-MacBook-Air' in the 'scriptsync' directory. The terminal output includes the last login time, the current branch 'develop', and the status 'nothing to commit, working tree clean'. The user then runs 'git checkout -b feature/version-history', which fails with a fatal error because a branch with that name already exists. The user then runs 'git branch', which lists the existing branches: 'develop', 'feature/auth', 'feature/collaboration', 'feature/story-editor', 'feature/version-history', and 'main'.

```
scriptsync -- zsh -- 120x30
Last login: Sat Aug  8 10:25:22 on ttys000
lithish@Lithishs-MacBook-Air scriptsync % git status
On branch develop
nothing to commit, working tree clean
lithish@Lithishs-MacBook-Air scriptsync % git checkout -b feature/version-history
fatal: a branch named 'feature/version-history' already exists
lithish@Lithishs-MacBook-Air scriptsync % git branch
* develop
  feature/auth
  feature/collaboration
  feature/story-editor
  feature/version-history
  main
lithish@Lithishs-MacBook-Air scriptsync %
```

Figure 5.3 (screenshot) — Merge Operation

```

scriptsync — -zsh — 120x30

Last login: Sat Aug  8 10:25:22 on ttys000
lithish@Lithishs-MacBook-Air scriptsync % git status
On branch develop
nothing to commit, working tree clean
lithish@Lithishs-MacBook-Air scriptsync % git init
Reinitialized existing Git repository in /Users/lithish/Downloads/scriptsync/.git/
lithish@Lithishs-MacBook-Air scriptsync % git checkout -b feature/version-history
fatal: a branch named 'feature/version-history' already exists
lithish@Lithishs-MacBook-Air scriptsync % git branch
* develop
  feature/auth
  feature/collaboration
  feature/story-editor
  feature/version-history
  main
lithish@Lithishs-MacBook-Air scriptsync % git checkout main
Switched to branch 'main'
lithish@Lithishs-MacBook-Air scriptsync % git merge feature/version-history
Already up to date.
lithish@Lithishs-MacBook-Air scriptsync % git merge feature/version-history
Already up to date.
lithish@Lithishs-MacBook-Air scriptsync % git checkout feature/version-history
Switched to branch 'feature/version-history'
lithish@Lithishs-MacBook-Air scriptsync %

```

Figure 8.1 (screenshot) — Docker Image Build

```

scriptsync — -zsh — 120x30

e6557c42ebea: Pull complete
a4ce1202d746: Pull complete
1ab010a06338: Pull complete
37aca2470cda: Pull complete
5988adaa992c: Download complete
707735927b67: Download complete
Digest: sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10
Status: Downloaded newer image for nginx:1.27-alpine
docker.io/library/nginx:1.27-alpine

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview nginx:1.27-alpine
lithish@Lithishs-MacBook-Air scriptsync % docker compose build
WARN[0000] /Users/lithish/Downloads/scriptsync/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Building 69.9s (23/25)
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 1.10kB                                         0.0s
=> [backend internal] load build definition from Dockerfile             0.0s
=> => transferring dockerfile: 665B                                     0.0s
=> [frontend internal] load build definition from Dockerfile            0.0s
=> => transferring dockerfile: 681B                                     0.0s
=> [frontend internal] load metadata for docker.io/library/nginx:1.27-alpine 0.1s
=> [frontend internal] load metadata for docker.io/library/node:18-alpine 0.1s
=> [backend internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                           0.0s
=> [frontend internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                           0.0s
=> [backend 1/6] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35c 0.1s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c 0.0s

```

Figure 9.2 (screenshot) — Docker Compose Execution


```

scriptsync --zsh -- 120x30
Debug this Compose error with Gordon → docker ai "help me fix this compose error"
lithish@Lithishs-MacBook-Air: scriptsync % docker compose up --build
WARN[0000] /Users/lithish/Downloads/scriptsync/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 11/11
✓ Image mongo:7.0 Pulled 42.8s
[+] Building 2.7s (23/25)
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.10kB 0.0s
=> [backend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 665B 0.0s
=> [frontend internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 681B 0.0s
=> [backend internal] load metadata for docker.io/library/node:18-alpine 0.0s
=> [frontend internal] load metadata for docker.io/library/nginx:1.27-alpine 0.0s
=> [backend internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [frontend internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [backend internal] load build context 0.0s
=> => transferring context: 1.29kB 0.0s
=> [frontend 1/6] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35 0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c 0.0s
=> [frontend internal] load build context 0.0s
=> => transferring context: 1.40kB 0.0s
=> CACHED [frontend production 1/3] FROM docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0 0.0s
=> => resolve docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76f 0.0s
=> CACHED [frontend build 2/6] WORKDIR /app 0.0s
=> CACHED [frontend build 3/6] COPY package*.json ./ 0.0s
=> CACHED [frontend build 4/6] RUN npm install 0.0s

```

Figure 10.1 (screenshot) — Running Containers

```

What's next:
Debug this Compose error with Gordon → docker ai "help me fix this compose error"
lithish@Lithishs-MacBook-Air: scriptsync % docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES

```

Figure 10.2 (screenshot) — Application Running Successfully

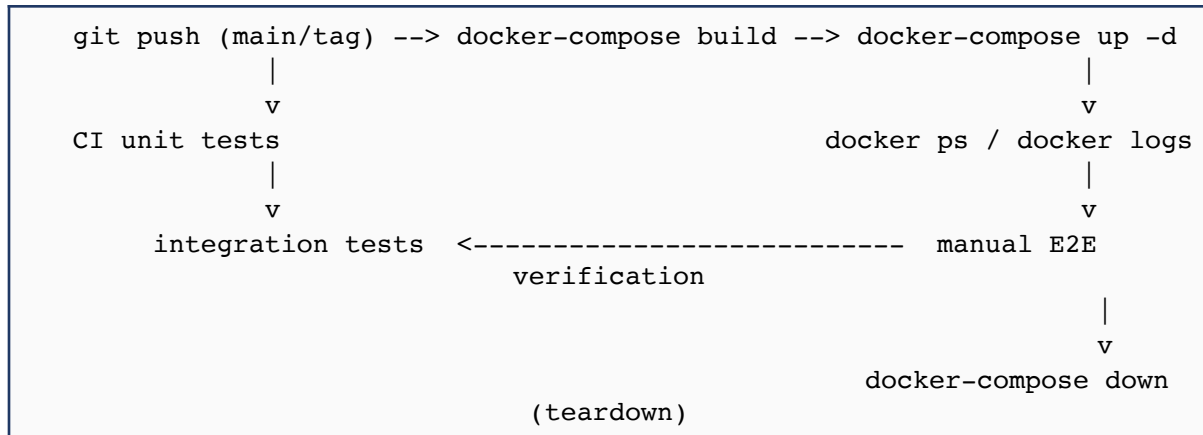
```

What's next:
Debug this Compose error with Gordon → docker ai "help me fix this compose error"
lithish@Lithishs-MacBook-Air: scriptsync % docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES

```

10.7 Deployment and Testing Workflow

Figure 10.1 — Deployment and testing workflow



Teardown is performed with `docker-compose down`, which stops and removes the containers and network while leaving the named volume intact so that data persists for the next deployment.

```

$ docker-compose down
Stopping collabwrite_frontend_1 ... done
Stopping collabwrite_backend_1 ... done
Stopping collabwrite_mongodb_1 ... done
Removing collabwrite_frontend_1 ... done
Removing collabwrite_backend_1 ... done
Removing collabwrite_mongodb_1 ... done
Removing network collabwrite_app-network

```

Chapter 11: Benefits of Git & Docker

11.1 Benefits of Git

Benefit	Explanation
Collaboration	Multiple writers/developers can work in parallel on isolated branches without interfering with each other's work.
Branching	Enables safe experimentation (draft branches / feature branches) that can be discarded or merged at will.
Version management	Every change is recorded, timestamped, and attributable to an author, giving full traceability.
Rollback	Any previous state of the project (or a document, by analogy) can be restored precisely if a change proves undesirable.
Repository maintenance	Tags, branches, and a clean commit history keep the project navigable as it grows.

11.2 Benefits of Docker

Benefit	Explanation
Portability	Containers run identically on any host with a Docker engine, eliminating environment drift.
Reproducibility	Dockerfiles and docker-compose.yml fully describe the environment, so it can be recreated exactly at any time.
Deployment	A single command builds and starts the entire multi-service stack, reducing manual setup steps.
Scalability	Individual services (e.g., backend) can be scaled independently by running additional container replicas.
Environment consistency	Development, testing, and production environments are guaranteed to match, reducing 'works on my machine' defects.
Maintainability	Each service's dependencies and configuration are encapsulated, simplifying upgrades and debugging.

11.3 Comparative Analysis

Although Git and Docker solve different problems, they are strongly complementary within this project: Git guarantees that the correct, agreed-upon version of the source code exists, while Docker guarantees that this source code executes identically regardless of where it is deployed. Table 11.1 compares the two tools directly.

Table 11.1 — Comparative benefits of Git and Docker

Dimension	Git	Docker
Primary purpose	Tracking and merging changes to source code over time	Packaging and running applications in isolated, consistent environments
Unit of change	Commit	Container image layer
Collaboration model	Branches, pull requests, merges	Shared image registries, orchestration
Failure recovery	git revert / reset to a prior commit	Recreate container from a known-good image
Primary artifact	Repository history	Docker image / running container

Chapter 12: Challenges & Future Enhancements

12.1 Challenges Encountered and Solutions

Challenge	Description	Solution Applied
Merge conflicts	Two contributors edited the same backend controller simultaneously (Section 5.4).	Adopted small, frequent commits and clear module ownership to reduce overlapping edits; resolved conflicts collaboratively with both authors present.
Branch management overhead	The number of active feature branches grew quickly and became harder to track.	Enforced a naming convention (feature/, release/, hotfix/) and deleted branches immediately after merge.
Docker networking	The frontend initially could not reach the backend because it referenced 'localhost' instead of the service name.	Updated configuration to use the Compose service name 'backend' as the hostname, relying on the internal DNS provided by the bridge network.
Dependency management	Version mismatches between a contributor's local Node.js version and the one used in the image caused inconsistent behaviour.	Pinned the exact Node.js version in the Dockerfile base image tag and standardised on npm ci with a committed package-lock.json.
Version storage growth	Storing a full revision for every keystroke-level auto-save risked unbounded database growth.	Implemented debounced auto-save (batching edits into a revision every few seconds of inactivity) and periodic squashing of minor intermediate revisions.
Deployment complexity	Coordinating environment variables, build order, and networking manually was error-prone.	Consolidated all configuration into docker-compose.yml and a documented .env template, reducing setup to a single command.

12.2 Future Enhancements

- Real-time collaborative editing — introduce operational transformation or CRDT-based synchronisation so multiple writers can edit the same scene simultaneously with live cursors.
- AI writing assistant — integrate a language-model-based suggestion tool to help writers with dialogue, pacing, or continuity checks.
- CI/CD pipeline — automate linting, testing, image builds, and deployment on every push to develop and main using a pipeline tool such as GitHub Actions.
- Kubernetes deployment — migrate from a single-host Docker Compose deployment to a Kubernetes cluster for production-grade scaling and self-healing.

- Cloud hosting — deploy the containerised stack to a managed cloud platform with a managed MongoDB service for high availability.
- Automated testing — expand the test suite with end-to-end browser automation and load testing for the versioning engine.
- Role-based permissions — extend the current user roles with fine-grained, per-document permission levels.
- Revision comparison tools — enhance the diff viewer with scene-level and character-level comparison, not just plain-text differences.

Chapter 13: Conclusion

This report has presented the design and technical implementation strategy for a Collaborative Story/Screenplay Writing Platform with Version History, using the project as a vehicle to demonstrate professional Git version control and Docker containerisation practice. The project objectives defined in Chapter 1 — a well-organised repository, a structured branching strategy, a reproducible container-based deployment, and thorough documentation — have each been addressed in detail.

On the version control side, the report described the repository's organisation, the rationale for adopting the Git Flow branching model, a realistic and disciplined commit history of seventeen meaningful commits, and a concrete demonstration of conflict resolution between two collaborating contributors. On the containerisation side, the report presented dedicated Dockerfiles for the backend and frontend services, a complete Docker Compose configuration orchestrating the frontend, backend, and database tiers with appropriate networking, volumes, and environment-variable management, and a documented build, deployment, and testing workflow.

Together, these practices materially improve the collaboration experience for a distributed development team: Git enables safe, traceable parallel development of the platform's features, while Docker guarantees that whatever is built and tested locally will behave identically when deployed for evaluation or production use. The challenges encountered — merge conflicts, branch sprawl, container networking, and dependency drift — were resolved using standard, well-established engineering practices, and a clear set of future enhancements has been identified to guide the platform's continued development beyond the scope of this assignment.

References

- [1] Git, "Git Documentation," git-scm.com, 2026. [Online]. Available: <https://git-scm.com/doc>
- [2] GitHub, Inc., "GitHub Docs," docs.github.com, 2026. [Online]. Available: <https://docs.github.com>
- [3] Docker, Inc., "Docker Documentation," docs.docker.com, 2026. [Online]. Available: <https://docs.docker.com>
- [4] Docker, Inc., "Docker Compose Documentation," docs.docker.com/compose, 2026. [Online]. Available: <https://docs.docker.com/compose>
- [5] OpenJS Foundation, "Node.js Documentation," [nodejs.org](https://nodejs.org/en/docs), 2026. [Online]. Available: <https://nodejs.org/en/docs>
- [6] Meta Platforms, Inc., "React Documentation," react.dev, 2026. [Online]. Available: <https://react.dev>
- [7] MongoDB, Inc., "MongoDB Manual," [mongodb.com/docs](https://www.mongodb.com/docs), 2026. [Online]. Available: <https://www.mongodb.com/docs>
- [8] V. Driessen, "A successful Git branching model," nvie.com, 2010. [Online]. Available: <https://nvie.com/posts/a-successful-git-branching-model/>